

FreelanceHub

A MERN freelance marketplace for posting work, hiring talent, chatting, holding escrow (full job, milestones, or hourly hours), cancelling with refunds, and completing jobs with reviews.

Tests, CI, Playwright, and Docker are part of the repo; Stripe, S3, Redis, and SMTP stay optional.

React 19 + Vite 6

Express + MongoDB

Socket.IO

httpOnly JWT

CI + Docker

Repository: github.com/ShadowAN-AB/Freelance_Marketplace

Local app: <http://127.0.0.1:5178> or <http://localhost:5178> · **API:** <http://localhost:5001> · **Compose web:** <http://localhost:8080>

Document date: 21 September 2026 · Version 2.4

Contents

1. 1. What this project is
2. 2. Goals and non-goals
3. 3. Technology stack
4. 4. System architecture
5. 5. Repository layout
6. 6. Domain model
7. 7. Roles and screens
8. 8. Core hiring loop
9. 9. Feature catalogue
10. 10. Client application
11. 11. REST API
12. 12. Real-time chat and presence
13. 13. Security, auth, and validation
14. 14. Tests, CI, and quality
15. 15. Local setup and demo
16. 16. Operations and production hooks
17. 17. Viva talking points
18. 18. Presentation sprint (v2.2)
19. 19. Worthy viva sprint (v2.3)
20. 20. LLM matching and service split (v2.4)

1. What this project is

FreelanceHub is a full-stack marketplace where **clients** post paid projects and **freelancers** bid. After a hire, the two parties chat, the freelancer submits work (optionally with files), the client can request revisions, cancel with an escrow refund, or approve. Simulated escrow is released on complete. Both sides can then leave a review. An **admin** account can inspect users, projects, contracts/escrow, reports, and an audit log.

The product is a classic MERN monolith (MongoDB, Express, React, Node.js). The demo holds money as one `Payment` per contract: hire creates `held`, complete or milestone/hour approval increments `releasedAmount`, cancel sets `refunded`. Stripe, object storage, Redis, and SMTP are env-optional. If they are unset, the same hire loop still works.

One-line summary. Post (fixed, milestones, or hourly) → propose (or rebid after withdraw) → hire (escrow hold) → chat → submit work → optional revision *or* cancel/refund → release (full, slice, or hours) → review.

2. Goals and non-goals

In scope

- Email/password auth with httpOnly cookies, refresh tokens, and Bearer JWT
- Email verify and password reset (JSON mailer or SMTP)
- Project posting, URL-backed search, skill matching
- One pending proposal per freelancer per project; rebid after withdraw
- Hire, contract, escrow hold / release / refund
- Fixed-price milestones with partial release, or hourly time against a budget cap
- Invite-to-bid, chat attachments, saved searches, CSRF on cookie sessions
- Deliverable uploads (disk or optional S3)
- Project-scoped chat with join auth, multi-tab presence, typing
- Saved items, shortlists, reports, verified-skill tags
- Public client pages and richer freelancer portfolios
- Admin analytics, contracts/escrow, audit log (no impersonation)
- API tests, GitHub Actions CI, lint, Docker Compose
- Optional OpenAI-compatible LLM match rationales (skill overlap always works)
- Compose split: gateway + auth + marketplace + realtime (local `npm run dev` stays one process)

Out of scope (still)

- Google / social login
- Generated cover letters
- Native mobile apps
- Multi-currency
- Mandatory Stripe — it is a vertical slice behind env vars, not the default demo

3. Technology stack

Layer	Choice	Why it is used
Client	React 19, Vite 6, Tailwind CSS 4	SPA UI, fast local dev, utility styling
Routing / data	React Router 7, TanStack Query, Axios	URL filters, cached REST, <code>withCredentials</code>
Realtime client	Socket.IO client	Messages, presence, typing
Charts	Recharts	Admin analytics
API	Node.js, Express 4	REST + static <code>/uploads</code>
Data	MongoDB 7, Mongoose 8	Users, projects, contracts, audit log
Auth	JWT access + refresh, bcryptjs, httpOnly cookies	Session cookies plus Bearer for sockets
Validation	Zod	Request body schemas
Uploads	Multer + optional S3	Avatars 2 MB; deliverables 8 MB
Realtime server	Socket.IO + optional Redis adapter	Rooms, presence sets, multi-process
Mail	Nodemailer	JSON transport in demo; SMTP when set
Quality	node:test + supertest, Vitest, ESLint, Prettier	Hire-loop API tests and client helpers
Hardening	Helmet, CORS, request ids, rate limits	Baseline HTTP headers and logs

Local ports avoid macOS conflicts: API **5001**, Vite **5178**. Vite proxies `/api`, `/uploads`, `/health`, and `/socket.io` to the API. `CLIENT_URL` is required; the process will not start without it.

4. System architecture

The browser never talks to MongoDB. It talks to Express over HTTP (JSON and multipart, cookies included) and to the same Node process over WebSockets. Access tokens travel as httpOnly cookie `fh_access` and/or `Authorization: Bearer`. Refresh uses cookie `fh_refresh` (or a JSON body). Socket.IO authenticates the handshake token or the access cookie.

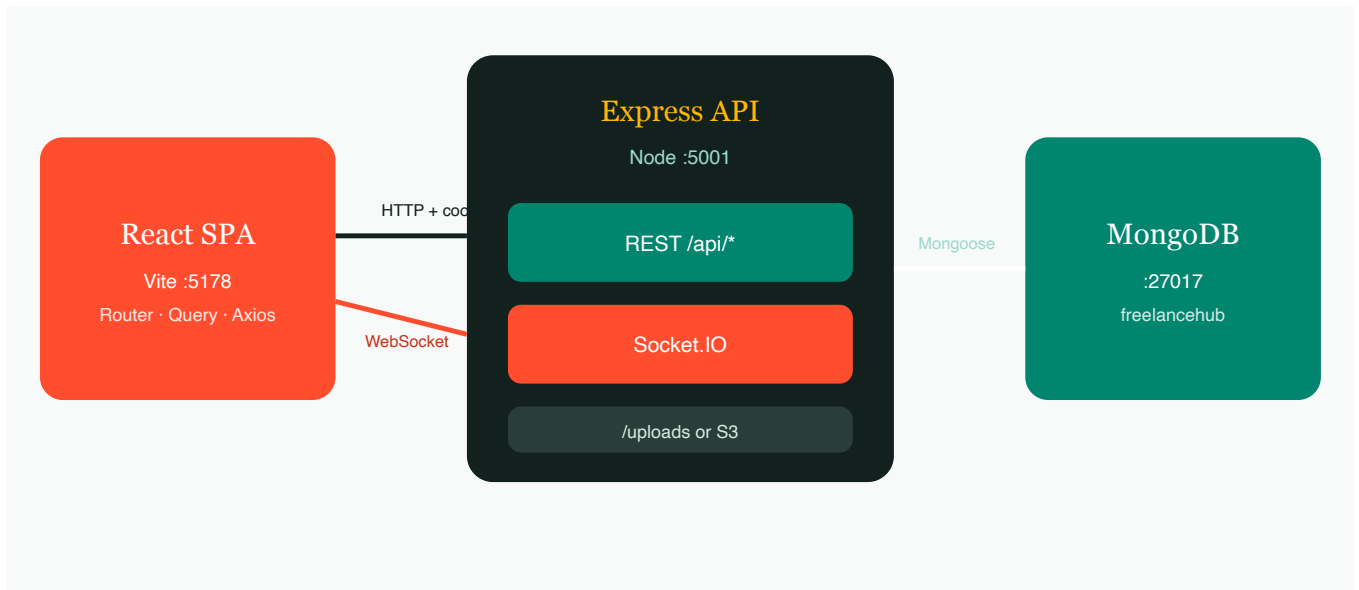


Figure 1. Browser SPA, single Express process (REST + Socket.IO + files), MongoDB. Optional Redis/S3/SMTP/Stripe attach via env.

On load, AuthContext calls `GET /api/auth/me` (cookies or stored Bearer token). A 401 triggers `POST /api/auth/refresh` ; if that fails the UI signs out. Toasts replace `alert()` for proposal success and work actions.

Local development still runs one Node process (`SERVICE=all` , default). Docker Compose starts four Node containers: **auth** (login, users), **marketplace** (projects, hire, escrow, admin), **realtime** (chat, notifications, Socket.IO), and a **gateway** on :5001 that proxies by path. They share Mongo and `JWT_SECRET` . Tests always hit the composed app so the hire loop does not depend on Docker.

5. Repository layout

```
Freelance_Marketplace/
├── client/                    React SPA (Vite)
│   ├── src/App.jsx           Routes, ToastProvider, role guards
│   ├── src/context/          Auth, toasts, 401 refresh
│   ├── src/layouts/          Public nav and signed-in shell
│   ├── src/pages/            public, auth, client, freelancer, admin, shared
│   ├── Dockerfile            nginx + /api proxy
│   └── vite.config.js         Proxies → :5001 · Vitest
├── server/
│   ├── src/server.js         createApp() + start(); health checks Mongo
│   ├── src/models/           Including AuditLog
│   ├── src/controllers/      Route handlers
│   ├── src/routes/           Express routers
│   ├── src/middleware/       JWT/cookies, Zod, Multer
│   ├── src/services/         socket, mailer, storage, payments, audit, logger
│   ├── src/seeds/seed.js      Demo data (refuses NODE_ENV=production)
│   ├── test/api.test.js      Hire-loop supertest suite
│   └── uploads/              Local files when S3 is unset
├── .github/workflows/ci.yml
├── docker-compose.yml        mongo + api + web
├── docs/                     This HTML + PDF
└── README.md
```

A native `mongod` on port 27017 can hide an empty Docker volume. The API uses whatever is on `MONGO_URI`.

6. Domain model

Collections are linked by ObjectIds. A project has at most one hired proposal. A contract is unique per project. A payment is unique per contract. Chat is unique per (project, unordered pair of users). Admin actions write `AuditLog` rows (blocks, report updates, skill verification). Impersonation is disabled.

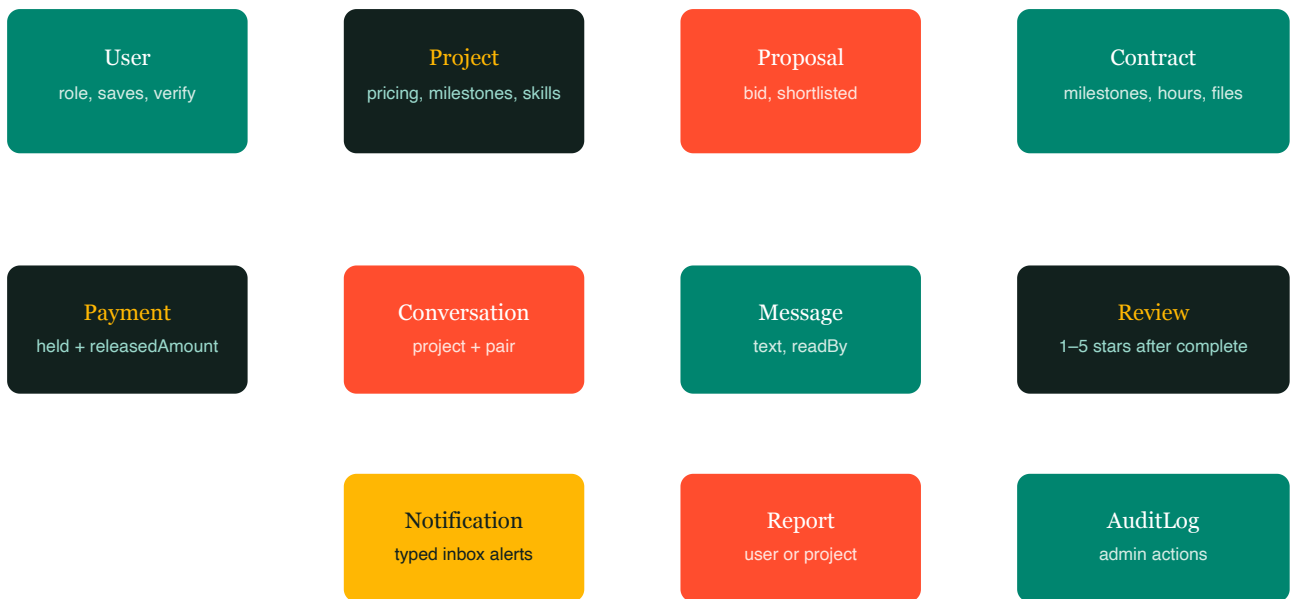


Figure 2. Main collections plus audit log. Users also hold saved ids, refresh-token hash, and verified skills.

Important statuses

Entity	Statuses
User.role	freelancer , client , admin
Project	open → in_progress → completed (or cancelled)
Proposal	pending , accepted , rejected , withdrawn (withdrawn may become pending again)
Contract	active , completed , cancelled (stores disputeReason)
Project.pricingType	fixed (optional 2–3 planned milestones) or hourly
Payment	held , released , refunded plus releasedAmount for partial milestone/hour payouts
Contract.milestones	pending → submitted → released (revision returns to pending)
Report	open , reviewed , dismissed

7. Roles and screens

Role	Can do	Primary screens
Guest	Browse projects and talent, open public client pages. Must log in to bid, save, report, or chat.	Home, /projects, /freelancers, /clients/:id, login, register, forgot/reset password
Freelancer	Propose on open projects (form hides after a pending/accepted bid; rebid after withdraw), save projects, submit work, cancel an active contract, chat after a proposal exists, review after completion.	Dashboard, Browse, Proposals, Active work, Saved, Messages, Earnings, Profile, Settings
Client	Post/edit/cancel open projects (fixed, milestones, or hourly), invite talent, shortlist and accept/reject, save talent, request revision, release a milestone or hours, or cancel/refund, report, chat after a proposal exists.	Dashboard, Post project, My projects, Proposals inbox, Active work, Saved, Messages, Profile, Settings
Admin	Stats, block users, verify freelancer skills, list projects, open contracts/escrow, read the audit log, resolve reports. Seeded only — not self-registered.	/admin, Users, Projects, Reports, Contracts, Audit

8. Core hiring loop

This is the money path. Saved items, shortlists, reports, and presence sit on top of it.

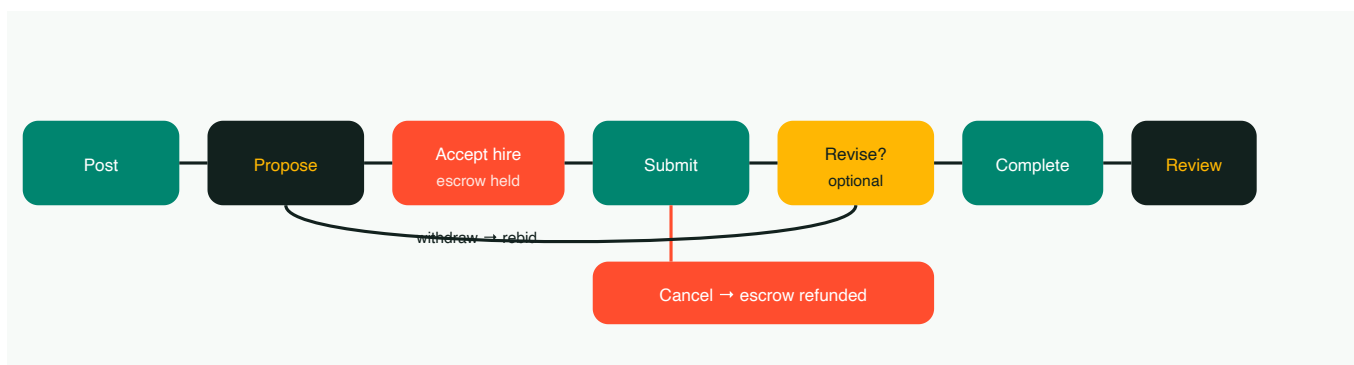


Figure 3. Happy path, revision loop, withdraw/rebid, and cancel/refund. Complete still requires a current `workSubmittedAt`.

Domain rules (enforced on the server)

- Unique index on `{ projectId, freelancerId }`. A second pending bid is 409. A `withdrawn` row is updated back to `pending`.
- You cannot bid on your own project or on a non-open project. After a proposal exists, the project detail page shows status instead of a second form.
- Accepting a bid rejects other pending bids, sets the project to `in_progress`, creates a contract (copying planned milestones, or a single “Full project” slice, or an hourly cap at `budgetMax`) and a held payment, and opens a conversation.

- Chat requires a pending or accepted proposal. Socket `conversation:join` is participant-checked (join is not an open room id).
- Only the hired freelancer can submit work. Only the client can request a revision or complete.
- Either party (or admin) can cancel an `active` contract with a reason; held escrow becomes `refunded` and the project is cancelled.
- Revision keeps status `active`, increments `revisionCount`, stores `revisionNote`, and clears `workSubmittedAt`.
- Reviews are allowed only after `completed`, once per reviewer per contract.

9. Feature catalogue

9.1 Auth and profiles

Register is limited to `freelancer` or `client`. Passwords are hashed with bcrypt (cost 12). Login is rate-limited (40 / 15 minutes). Sessions set httpOnly cookies and return a Bearer token for sockets. `POST /auth/refresh` rotates refresh hashes. `POST /auth/logout` clears cookies even if the access token already expired. Email verification and password reset send mail through Nodemailer (JSON log in demo). `SKIP_EMAIL_VERIFY=true` (and `NODE_ENV=test`) skip the verify gate for local/seeded accounts.

Profiles store bio, location, avatar, freelancer skills/rate/portfolio images/availability, verified-skill tags (admin-set), or client company name. Public client pages list that company's projects.

9.2 Projects and search

Browse keeps `q`, category, pricing type (fixed/hourly), due-soon, sort (newest / highest budget / due soonest), and page in the URL. Cards show **Fixed price**, **Fixed · N milestones**, or **Hourly**, plus pending bid counts and a relative due date (`due in 3d` / `overdue 2d`). Project detail lists the planned milestone table and a **Similar open work** strip from `GET /projects/:id/similar` (same category or overlapping skills). Clients see pending bid counts on My projects and can **Duplicate** any brief into a fresh open listing (new deadline, no hire). Talent browse keeps hourly-rate bounds in the URL. Matching score is skill overlap, capped at 100%.

9.3 Proposals and shortlist

Cover letter (counted out of 4000), bid amount, estimated days. The bid form warns when the amount is above `budgetMax`. Clients star pending proposals. The compare table can sort by shortlist, lowest bid, fastest delivery, or skill match, and flags over-budget rows. Pending rows have Accept, Reject, and Message. After a pending bid, either party can `POST /conversations` with `{ projectId, userId }` and land on `/app/messages/:id`. Outsiders still get 403. **Suggested talent** ranks freelancers with `matchScore` and a one-line rationale. If `LLM_API_KEY` is set the rationale is written by an OpenAI-compatible model; otherwise the server writes it from overlap and gaps. Tests force the heuristic engine.

9.4 Work, deliverables, revisions, disputes

Submit-work is multipart (PDF, ZIP, PNG, JPEG, WebP, 8 MB, up to 8 files). Files persist to disk or S3. Notifications, dashboard cards, and My projects “in progress” rows deep-link to

`/app/work/:id` so the matching contract expands. The work card shows an escrow progress bar (`releasedAmount / amount` plus “1 of 2 slices”). The client types a per-slice revision note instead of a hardcoded string. Approve the whole job, release one milestone, approve hourly time against the escrow cap, or Cancel & refund remaining escrow. Partial payouts live on

`Payment.releasedAmount` — there is still one payment row per contract. Ledger rows on Payments / Earnings link to `/app/work/:id/invoice`.

9.5 Saved items and saved searches

Freelancers bookmark projects; clients bookmark talent. `/app/saved` lists them. Named saved searches replay browse filters from the dashboard.

9.6 Messaging, unread, presence, typing

Unread count is threads whose latest message is not in `readBy`. Presence is a **set of socket ids per user** so a second tab does not kick the first offline. Chat notifications fire only if the other party is offline. Typing events stay inside the conversation room.

9.7 Notifications

Proposal received/accepted/rejected, work submitted, revision requested, contract completed, contract cancelled, review received, report updates. Alerts use a keyboard-focusable disclosure and `aria-live` unread counts. The inbox can **Mark all read**. Success toasts use `aria-live="polite"`.

9.8 Reports and admin

Users report a user or project. Admins review or dismiss (audited). Admins block users (audited), verify skills (audited), and open contracts with escrow status. There is no impersonate control.

9.9 Desk extras (presentation pack)

Project edit, proposal compare table, notifications inbox with type filter, invite-to-bid, chat attachments, printable invoice at `/app/work/:id/invoice`, CSRF token cookie `fh_csrf` (skipped for Bearer and GET), and a Playwright hire-loop smoke in CI.

10. Client application

Path	Who	Page
/	Public	Home plus five-minute viva beat and demo accounts
/projects , /projects/:id	Public	URL filters including pricing type; milestone plan; bid form or “your proposal”
/freelancers , /freelancers/:id	Public	Hourly-rate filters, verified skills, portfolio images
/clients/:id	Public	Client company page and their projects
/login	Guest	One-click Priya / Aisha / Admin fill
/register , /forgot-password , /reset-password , /verify-email	Guest	Auth and recovery
/app/dashboard	Signed in	Due-soon, saved searches, client “Active work” with Scan MVP ready to release
/app/work	Both	Milestones, hours, files, revision, release, cancel/refund, review; nav badge for pending slices
/app/work/:id	Both	Same desk, focused on that contract (escrow bar, Message, Approve hours)
/app/work/:id/invoice	Both	Printable invoice of held / released / refunded
/app/notifications	Signed in	Typed inbox; hire/submit links go to /app/work/:id
/app/messages , /app/messages/:id	Both	Threads with counterpart + project title; presence, typing
/app/projects	Client	Open / In progress / Done chips; in-progress rows link Active work
unknown public URL	Public	404 with links home / projects / demo login
/admin/contracts , /admin/audit	Admin	Escrow list and action log

Visual language is unchanged: mint paper, coral buttons, teal accents, saffron highlights. Nested dashboard routes render through an `Outlet` in `AppShell`.

11. REST API

Base URL in the browser is `/api` (proxied). JSON errors: `{ message, errors? }`. List endpoints for contracts and conversations are paginated (`page` , `limit` , `pages` , `total`). Messages already paginated.

Method	Path	Auth	Purpose
POST	/auth/register, /auth/login	Public (limited)	Cookies + JWT
POST	/auth/refresh, /auth/logout	Cookie / optional	Rotate or clear session
GET	/auth/me	User	Current user
GET / POST	/auth/verify-email, /forgot-password, /reset-password	Public	Email flows
PATCH	/auth/password	User	Change password
GET	/stats	Public	Open projects, freelancers, completed jobs, escrow held
GET	/projects	Optional	List/filter by q, category, dueSoon, pricingType, sort; each row has proposalCount
GET	/projects/:id	Optional	Project + myProposal for the bidder
GET	/projects/:id/similar	Optional	Nearby open listings by category or skills
POST	/projects/:id/duplicate	Owner	Fresh open copy of the brief
POST	/contracts/:id/milestones/:mid/release	Client	Partial escrow release
POST	/contracts/:id/time-entries/:tid/approve	Client	Hourly hours against the cap
POST	/contracts/:id/cancel	Party	{ reason } → refund held escrow
GET	/payments/me, /payments/me.csv, /payments/mode	User	Ledger JSON or CSV; simulated vs stripe
POST	/payments/webhook	Stripe signature	Ignored unless secrets are set
POST	/conversations	Party	Open a thread after a pending/accepted bid; outsider 403
POST	/proposals/:id/reject	Client	Decline a pending bid
POST	/contracts/:id/reviews	Party	1–5 stars after complete
GET	/users/:id/reviews	Public	Reviews left for that user
POST	/reports	User	Report a user or project
GET	/admin/stats, /admin/reports	Admin	Role/project/escrow counts; report queue

Method	Path	Auth	Purpose
GET	/admin/contracts, /admin/audit	Admin	Escrow and audit log
PATCH	/admin/users/:id/verify-skills	Admin	Verified skill tags
GET	/health	Public	{ ok, mongo, service } — 503 if Mongo is down

Earlier catalogue routes (proposals, submit-work, revision, complete, conversations, notifications, reports) are unchanged in purpose; see the source routers for the full list.

12. Real-time chat and presence

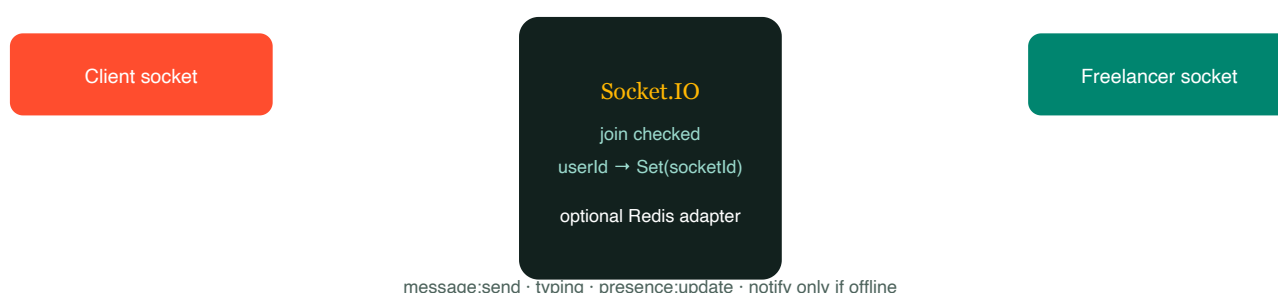


Figure 4. Join is authorized. Presence is a set of sockets per user. Chat alerts are quiet when the other party is online.

Event	Direction	Meaning
conversation:join	Client → server	Join only if the user is a participant
typing	Both	In-room typing indicator
message:send	Client → server	Persist, broadcast; notify if peer offline
message:new / message:read	Server → room	Live thread and receipts
presence:update	Server → all	Online only when the last tab disconnects

13. Security, auth, and validation

- Passwords bcrypt-hashed; secret fields stripped from public user payloads.
- Access cookie `fh_access` (httpOnly, SameSite lax, Secure in production) plus refresh cookie `fh_refresh`. Refresh token is stored hashed on the user.
- Axios sends cookies and Bearer; 401 retries refresh once, then clears the UI session.
- CORS origin is exactly `CLIENT_URL` with credentials. Socket.IO uses the same origin.
- Zod on mutating routes. Multer rejects disallowed types. JSON body 1 MB. Auth rate limits skipped only in tests.

- Request id header `X-Request-Id` on every response. Structured JSON logs.
- CSRF: mutating cookie requests need header `X-CSRF-Token` matching cookie `fh_csrf`. Bearer JWT and GET/HEAD skip the check so API tests stay green.

Demo secrets. Rotate `JWT_SECRET` before any shared deploy. Disk `server/uploads/` does not survive multiple API instances — set S3 for that. Payments are simulated unless Stripe env vars are set.

14. Tests, CI, and quality

`cd server && npm test` runs node:test + supertest against `freelancehub_test` (or `MONGO_URI_TEST`). The hire-loop suite is unchanged, plus heuristic match rationales, gateway path routing, and an auth-only app that 404s marketplace routes. Matching tests never call an LLM (`NODE_ENV=test` disables it).

`cd client && npm test` runs Vitest on format helpers (INR, pricing labels, relative times, due dates, escrow progress). `cd client && npm run e2e` is Playwright: a hire-loop smoke (register → post → bid → accept → submit) plus viva smokes (home copy, hourly filter, 404, Priya dashboard Scan MVP). GitHub Actions on `main` starts Mongo 7, runs server lint/test, then client lint/test/build, then `npm --prefix server run seed` against `freelancehub_e2e` before Playwright so demo listings exist. CI starts the monolith (`SERVICE=all`), not the four Compose workers.

15. Local setup and demo

Prerequisites

- Node.js 20+ (tested on 22)
- MongoDB on `127.0.0.1:27017` — Docker `mongo` service or local `mongod`

```
cp .env.example server/.env
cd server && npm install && npm run seed && npm run dev
cd client && npm install && npm run dev
```

Open <http://127.0.0.1:5178> or <http://localhost:5178>. Health: <http://localhost:5001/health> (includes `mongo: true`). Keep `SKIP_EMAIL_VERIFY=true` in `server/.env` for the viva.

```
JWT_SECRET=replace-with-a-long-random-string docker compose up --build
```

Compose serves nginx on <http://localhost:8080> and proxies API/socket/uploads to the **gateway** (`api:5001`). Workers listen on 5002–5004. Do not run `npm run seed` in production — the seed script exits if `NODE_ENV=production`. Local viva: keep using `npm run dev` as one process.

Every seeded account uses password `Password123!`.

Role	Email	Notes
Admin	admin@freelancehub.dev	Users, contracts, audit, reports
Client	priya@freelancehub.dev	Due-soon landing, Scan MVP submitted (real notes file), hourly desk with 4h pending
Client	arjun@freelancehub.dev	Completed SEO contract with Leo
Freelancer	aisha@freelancehub.dev	Pending logistics bid, inventory contract, hourly pairing desk; verified React/Node tags
Freelancer	kabir@..., meera@..., leo@...	Extra bids and a completed job

Ten-minute walkthrough

1. Home page live stats (open jobs, talent, completed work, escrow held) → **Viva demo login** → Priya chip → Sign in.
2. Dashboard: due-soon landing, Active work says **Ready to release · Scan MVP** (deep-links into that contract).
3. Open Active work. Download Scan MVP notes. Release Scan MVP. Escrow bar moves; Offline sync stays held.
4. On the hourly card, **Approve 4h**. Payments ledger rows open the invoice.
5. Message Aisha from the work card or the proposals table. Thread header shows counterpart + project title.
6. Log out. Login as Aisha. Open the logistics dashboard — **Your proposal / Pending**, not a second form. Browse `/projects?pricingType=hourly` for the still-open Node pairing listing.
7. As admin, open Contracts, Reports, and Audit (hire / milestone submit / report rows are seeded).

16. Operations and production hooks

Concern	Demo default	Production hook
Auth	Cookies + Bearer, 7d access in <code>.env</code>	Shorter <code>JWT_EXPIRES_IN</code> , unique <code>JWT_SECRET</code>
Mail	Nodemailer JSON transport (logged)	<code>SMTP_URL</code> , <code>MAIL_FROM</code>
Files	<code>server/uploads/</code>	<code>AWS_S3_BUCKET</code> (+ region / public URL)
Presence	In-memory socket id sets	<code>REDIS_URL</code> → Socket.IO Redis adapter
Payments	One <code>Payment</code> per contract; <code>releasedAmount</code> for slices	<code>STRIPE_SECRET_KEY</code> + webhook secret
Health	<code>/health</code> 200/503 from Mongo <code>readyState</code>	Load balancer on that URL

- Seed wipes marketplace collections. Never point it at a live database.
- Vite 6 is pinned because newer Vite/Rolldown builds failed on this machine's native bindings.
- If port 27017 already has a local MongoDB, Docker's mapped port will fail or you will see an empty database. Point `MONGO_URI` at the instance that contains the seed.

Source of truth. This document describes the codebase as of 21 September 2026 on `main`, version 2.4: worthy viva sprint plus optional LLM match rationales and a Compose gateway split (auth / marketplace / realtime).

FreelanceHub · MERN marketplace · github.com/ShadowAN-AB/Freelance_Marketplace

17. Viva talking points

Memorise these answers. Examiners usually ask “why”, not “show me another screen”.

Likely question	What to say
Why split the API?	Local <code>npm run dev</code> is still one process so the viva never needs four terminals. Compose sets <code>SERVICE=auth marketplace realtime</code> behind a gateway. Shared Mongo and JWT. Health on <code>:5001</code> lists which worker is up.
Is matching “AI”?	Ranking is deterministic skill overlap. The one-line reason uses an LLM only when <code>LLM_API_KEY</code> is set. Without a key you still see overlap % and a written gap list. Tests never call a model.
Is the money real?	No. Hire writes a <code>Payment</code> with status <code>held</code> . Complete or milestone/hour approval increments <code>releasedAmount</code> . Cancel sets <code>refunded</code> . Stripe is behind env vars for a later slice.
Why cookies and Bearer?	The SPA sends <code>httpOnly</code> cookies so XSS cannot steal the access token. Socket.IO and Playwright still use <code>Authorization: Bearer</code> . CSRF is required only for cookie POSTs; Bearer skips it.
How do you stop double bids?	Unique index on <code>{ projectId, freelancerId }</code> . A second pending bid is 409. Withdraw sets <code>withdrawn</code> ; the same row can go back to <code>pending</code> .
Fixed vs hourly?	Fixed may split into 2–3 milestones that must sum to <code>budgetMax</code> . Hourly holds <code>budgetMax</code> as a cap; approved hours cannot exceed remaining escrow.
How is chat authorised?	HTTP messages and socket <code>conversation:join</code> both check that the user is a participant. Join is not an open room id.
What is tested?	Hire-loop API tests plus matching rationales, gateway path routing, and <code>SERVICE=auth</code> 404s on marketplace routes. Vitest for INR/pricing/escrow progress. Playwright hire-loop plus seeded viva smokes. GitHub Actions seeds <code>freelancehub_e2e</code> before Playwright.
What would you do next if this shipped?	Rotate <code>JWT_SECRET</code> , turn off <code>SKIP_EMAIL_VERIFY</code> , put files on S3, turn on Stripe, put Socket.IO on Redis if you run more than one API process.

Demo sentence. “Priya hired Aisha on a two-slice inventory app. Scan MVP is already submitted. I will release that slice, leave Offline sync held, and show that one Payment row tracks both amounts.”

18. Presentation sprint (v2.2)

Added for the viva after the desk pack. Each item has a GitHub commit on `main` and, where it is an API change, a `node:test` case.

Surface	What to show
Home	Live <code>GET /api/stats</code> widgets: open projects, freelancers, completed jobs, escrow held.
My projects	Pending bid counts; Duplicate copies budget, skills, and milestones into a new open listing.
Project detail	Similar open work; cover-letter counter; over-budget bid warning; relative due date.
Browse	Sort by newest, highest budget, or due soonest. Bid counts on cards.
Proposals	Sort by shortlist / lowest bid / fastest / skill match; over-budget flag.
Active work	Status chips: all, active, submitted, completed, cancelled. Relative activity times.
Earnings / Payments	Freelancer earnings and client payments share <code>GET /payments/me.csv</code> .
Notifications	Relative timestamps and Mark all read.

FreelanceHub · MERN marketplace · github.com/ShadowAN-AB/Freelance_Marketplace

19. Worthy viva sprint (v2.3)

Shipped after v2.2 so the demo does not break when an examiner clicks. Each slice is its own GitHub commit on `main`.

Slice	What to show
Seed	Scan MVP notes is a real <code>/uploads/scan-mvp-notes.txt</code> . Audit has hire/submit/report rows. Priya has a second active card: Warehouse Node pairing desk with 4h pending. The original hourly listing stays open for browse e2e.
Desk	<code>/app/work/:id</code> expands the contract. Escrow bar uses <code>releasedAmount / amount</code> plus slice counts. Active-work nav badge. My projects Open / In progress / Done.
Chat	Message from proposals and the work card (<code>POST /conversations</code>). Thread header: counterpart + project. Invite banner + toast. Per-slice revision textarea.
Quality	Admin stats, reject, reviews, reports, and the email-verify gate have API tests. CI seeds before Playwright. Payments ledger links to invoices.
Docs	This HTML + PDF v2.3 and the README five-minute beat (hourly approve + audit log).

Demo sentence (v2.3). “Priya’s dashboard jumps straight into Scan MVP. I will download the notes, release that slice, approve four hours on the hourly desk, message Aisha from the card, then open Admin → Audit so you can see the seeded hire and submit rows.”

FreelanceHub · MERN marketplace · github.com/ShadowAN-AB/Freelance_Marketplace

20. LLM matching and service split (v2.4)

Local demo is unchanged: one Express process. Docker Compose is the split you show if someone asks “where are the microservices?”

Piece	What to show
Matching	<code>GET /projects/:id/matches</code> returns <code>engine: heuristic llm</code> , overlap %, and a one-line rationale. Proposals inbox: Suggested talent. Freelancer dashboard recommendations use the same helper.
Gateway	<code>GET /health</code> on :5001 → <code>service: freelancehub-gateway</code> plus <code>backends.auth/marketplace/realtime</code> .
Auth worker	:5002 — login, register, <code>/api/users</code> .
Marketplace worker	:5003 — projects, hire, escrow, admin, reviews, uploads.
Realtime worker	:5004 — conversations, notifications, Socket.IO.

Honest limit. This is a process split in front of one Mongo database, not independently deployable data stores. JWT is shared. That is enough to show routing and failure isolation without breaking the hire loop.